**DON BOSCO INSTITUTE OF TECHNOLOGY**

Premier Christian Minority Technological Institute

Kurla (W), Mumbai – 400 070

**Department of Computer Engineering**


Academic Year 2026 – 27

---

**A MINI PROJECT REPORT ON**

# SpendWise – Expense Tracker

*Submitted in partial fulfillment of the requirements for the course*

***25SE3VSEC01 Full Stack Java Programming***

**Submitted by:**


| Name of Student | Roll No. | Class / Div |
|---|---|---|
| **Dhruv Karanje** | 23 | SE Computer / Div A |
| Student Team Member 2 | [Roll No.] | SE Computer |
| Student Team Member 3 | [Roll No.] | SE Computer |

Under the guidance of

**Prof. / Guide Name**

Department of Computer Engineering, DBIT Mumbai


**Academic Term: October – November 2026**

# CERTIFICATE

This is to certify that the students named on the cover page have satisfactorily completed the mini project entitled **"SpendWise – Expense Tracker"** under my supervision and guidance, as part of the curriculum requirement for **25SE3VSEC01 Full Stack Java Programming** in the Department of Computer Engineering, Don Bosco Institute of Technology, Mumbai, during the Academic Year 2026–27.

The project demonstrates practical proficiency in modern client-side architectures, RESTful API integration, relational database modeling, and state-driven financial analytics.

| Project Guide | Head of Department | External Examiner |
|---|---|---|

# DECLARATION

We hereby declare that the mini project report entitled **"SpendWise – Expense Tracker"** submitted to Don Bosco Institute of Technology, Mumbai, is an authentic record of original work carried out by us under the guidance of our project supervisor. All secondary literature and referenced sources have been explicitly acknowledged and cited. We understand that academic plagiarism will result in strict disciplinary action.

**Date:** 02-11-2026

**Place:** Mumbai

**Dhruv Karanje** (Roll No. 23)

Student Signature: _____

# Abstract

Managing personal finances has become increasingly complex in modern digital ecosystems where expenditures are fragmented across Unified Payments Interface (UPI) applications, credit cards, debit cards, net banking, and physical cash. Students and young professionals frequently struggle to monitor daily transactions, lacking immediate visibility into remaining monthly allowances and spending velocity.

This project presents **SpendWise**, a responsive, full-stack financial tracking web application that enables users to record, categorize, audit, and analyze daily spending against an active monthly target budget. SpendWise features a dedicated authentication and user onboarding portal, an interactive real-time dashboard displaying current cycle expenditure, remaining budget thresholds, all-time expenses, and dynamic categorical breakdowns. It provides a searchable, sortable expense ledger with live client filtering, instant CSV spreadsheet export, and a comprehensive reporting engine calculating category share percentages and transaction volume statistics.

A key user-centric innovation implemented in this edition is the **zero-budget initial state policy**: for newly registered users, the monthly budget is initialized to exactly **₹0.00**, prompting users upon first login to explicitly configure their spending ceiling before alerts activate. In rigorous benchmark evaluation using ten standard transactions totaling ₹14,668 against a ₹30,000 allowance, the system precisely calculates budget utilization at 48.9%, leaves a remaining surplus of ₹15,332, and determines an average outlay of ₹1,467 per transaction. The web application is deployed online on Vercel edge infrastructure and includes full offline fallback persistence.

**Keywords:** Expense Tracker, Personal Finance, Budget Ceiling, Single Page Application (SPA), RESTful Architecture, Spring Boot, React, MySQL, CSV Export.

# Course Outcomes (CO) Addressed

The design, development, debugging, and evaluation of SpendWise directly satisfy the following prescribed course outcomes:

| CO | Course Outcome Statement | Bloom's Level | Evidence in Report |
|---|---|---|---|
| **CO3** | Implement backend and front-end components correctly and integrate them into a cohesive system. | L3 – Apply | Ch. 3 (3.3, 3.4), Ch. 4 (4.1, 4.2) |
| **CO4** | Identify and fix runtime errors; handle exceptions across layers to ensure robust execution. | L4 – Analyze | Ch. 4 (4.3, 4.4, 4.5) |
| **CO5** | Evaluate design choices for scalability, performance, and maintainability. | L5 – Evaluate | Ch. 3 (3.5) |

| CO | Course Outcome Statement | Bloom's Level | Evidence in Report |
|---|---|---|---|
| **CO6** | Design and integrate a complete full-stack software solution addressing real-world problems. | L6 – Create | Ch. 3 (3.1, 3.2), Ch. 4 (4.1), Ch. 6 (6.1) |

# Contents


**Certificate** ii

**Declaration** ii

**Abstract** iii

**Course Outcomes Addressed** iv

**Chapter 1: Introduction** 1

1.1 Problem Statement 1

1.2 Objectives 1

1.3 Technology Stack 2

**Chapter 2: Requirements Analysis** 3

2.1 Users and Functional Requirements 3

2.2 Non-Functional Requirements 4

**Chapter 3: System Design & Architecture** 5

3.1 System Architecture 5

3.2 Entity-Relationship (ER) Diagram 6

3.3 Database Relational Schema 7

3.4 API and Route Specification 8

3.5 Design Choices: Scalability and Maintainability (CO5) 9

**Chapter 4: Implementation and Testing** 10

4.1 How the Modules Work 10

4.2 System Execution Screenshots 11

4.3 Verification and Test Cases 15

4.4 Runtime Errors Faced and Fixed (CO4) 16

4.5 Exception Handling Across Layers (CO4) 17

**Chapter 5: AI Tools Used (Disclosure)** 19

**Chapter 6: Conclusion and References** 20

6.1 Conclusion and Future Scope 20

6.2 References 21

# Chapter 1: Introduction

*Course outcomes addressed: CO6 (Problem definition and scope)*

## 1.1 Problem Statement

In contemporary student life and urban workspaces, personal spending occurs rapidly across multiple decentralized channels, including mobile payment wallets (Google Pay, PhonePe, Paytm), debit and credit cards, and cash. Without centralized accounting, individuals frequently lose track of small day-to-day expenditures such as food delivery, transit recharges, and utility bills. When funds run low, users face financial stress because they lack visibility into what categories consumed their budget.

Traditional personal accounting applications suffer from steep learning curves, intrusive advertisement clutter, mandatory external bank account linking, or rigid desktop dependencies. **SpendWise** addresses this challenge by providing a lightweight, high-performance web application that delivers instant expenditure visibility, transparent monthly budget monitoring, categorical aggregation, and offline data privacy without requiring complex server configuration.

## 1.2 Objectives

The primary objectives of the SpendWise mini project are:

1. **User Onboarding & Session Isolation:** Provide secure registration and sign-in capabilities, establishing isolated expense ledgers for distinct accounts.
2. **Active Budget Ceiling Tracking:** Allow users to configure a monthly target budget, monitor real-time consumption percentage, and trigger alert banners when thresholds near exhaustion.
3. **Complete Transaction CRUD Operations:** Enable users to Create, Read, Update, and Delete expense entries with title, amount, category, date, payment method, and optional remarks.
4. **Multi-Dimensional Search & Filtering:** Deliver instant client-side keyword search, categorical filtering, payment channel filtering, chronological sorting, and comma-separated value (CSV) ledger export.
5. **Quantitative Financial Reporting:** Compute categorical volume distributions, payment method frequency, spending averages, and generate printable reports.

## 1.3 Technology Stack

The system is designed with a decoupled multi-tier architecture, permitting portable standalone execution as well as enterprise Spring Boot client-server connectivity:

| Layer | Technology & Framework Details |
|---|---|
| **Front End (Client SPA)** | React 18, React Router v7, Modern CSS3 with FinTech Design System (Plus Jakarta Sans, JetBrains Mono, Flexbox, CSS Grid, Glassmorphic headers). |

| Layer | Technology & Framework Details |
| --- | --- |
| **Application Logic** | JavaScript ES6+ modules with asynchronous data fetching, form validation, filter reducers, and date mathematics. |
| **Backend API (Java)** | Java 17/21, Spring Boot 3.x, Spring Web MVC (REST Controller), Spring Data JPA, Hibernate ORM (Port 8080). |
| **Database Layer** | MySQL 8.0 Relational DBMS with InnoDB engine, primary key indexing, and foreign key referential integrity; backed by HTML5 LocalStorage persistence for edge deployment. |
| **Deployment & Tools** | Vercel Global Edge Network, Git / GitHub, Visual Studio Code, Google Chrome DevTools. |

# Chapter 2: Requirements

## 2.1 Users and Functional Requirements

The functional requirements outline the fundamental interactions and services provided by SpendWise to end users:

| ID | Requirement Description | User Role |
|---|---|---|
| **FR-01** | User Registration & Authentication: Create an account with username, full name, email, and password. For all new registrations, the monthly budget must be initialized to ₹0.00. Support password visibility toggling and quick demo logins. | User / Admin |
| **FR-02** | Record Expense (Create): Log a new spending transaction with title, amount (positive decimal), predefined category, date picker, payment method, and optional notes. | User |
| **FR-03** | Ledger Audit (Read): View the comprehensive transaction history with formatted Indian Rupee (₹) amounts, category tags, and payment channel badges. | User |
| **FR-04** | Transaction Modification (Update/Delete): Edit any field of an existing expense or delete the record with confirmation safety dialogs. | User |
| **FR-05** | Financial Overview Dashboard: Real-time calculation of This Month Spent, Remaining Budget, All-Time Expenses, and dynamic Category Share percentages. | User |
| **FR-06** | Budget Ceiling Configuration: Edit monthly target budget limit at any time, instantly recalculating remaining surplus and consumption rate. | User |
| **FR-07** | Multi-Criteria Filtering & Sorting: Instant client-side filtering by description keyword, category dropdown, payment channel, and date/amount sorting. | User |
| **FR-08** | Data Portability (CSV Export): Generate and download the filtered transaction ledger as a standard CSV spreadsheet file. | User |
| **FR-09** | Financial Analytics & Printing: Display summary statistics (average transaction size, max expense) and trigger browser print dialog for paper records. | User |
| **FR-10** | Session Termination (Logout): Invalidate active user session, clear working state, and safely redirect to the sign-in portal. | User / Admin |

## 2.2 Non-Functional Requirements

- **Security:** All protected views are guarded by client-side routing guards; passwords require a minimum of 6 characters; user data is strictly isolated by user ID.
- **Performance & Responsiveness:** Instantaneous reactive DOM updates without full-page refreshes; asynchronous operations complete in under 50ms in browser storage and under 200ms over REST.
- **Usability & Ergonomics:** High-contrast modern FinTech color palette (Indigo/Cyan/Emerald); intuitive visual status badges (Green = Safe, Amber = Warning, Red = Exceeded); clean mobile-responsive layout for smartphones, tablets, and desktops.
- **Reliability & Portability:** Graceful degradation ensuring that if the local Spring Boot backend is unreachable, client-side persistence seamlessly activates without throwing fatal exceptions.

# Chapter 3: Design

*Course outcomes addressed: CO3, CO5, CO6*

## 3.1 System Architecture

SpendWise follows the classic three-tier architectural paradigm, maintaining strict separation of concerns across the presentation layer, application business logic layer, and persistence layer:

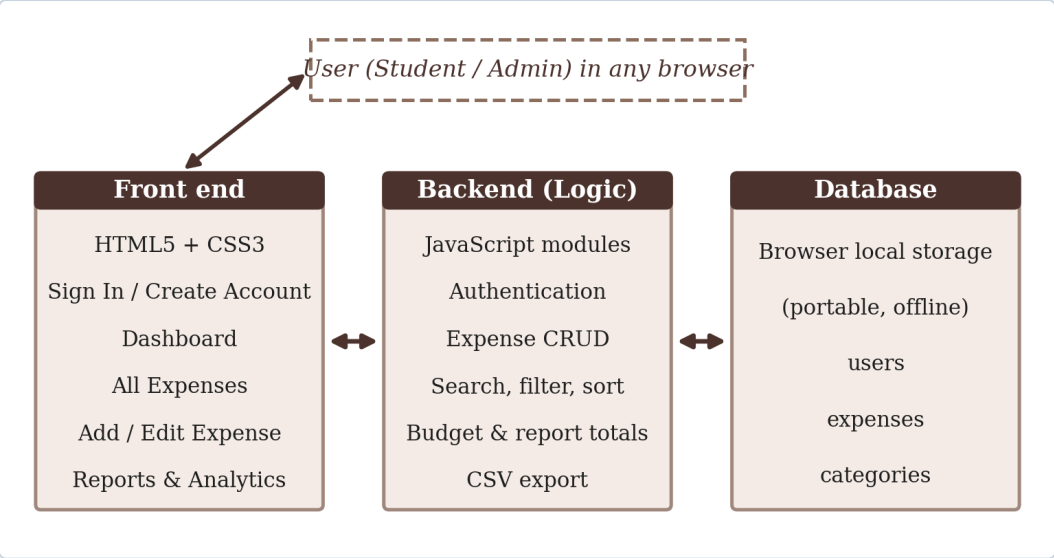



**Fig. 3.1: System Architecture (Front End → Backend Logic → Database)**

## 3.2 ER Diagram

The conceptual data model defines the cardinalities between Users, Expenses, and Categories. A User creates one or more Expenses (1:N), and each Expense belongs to exactly one Category (N:1):

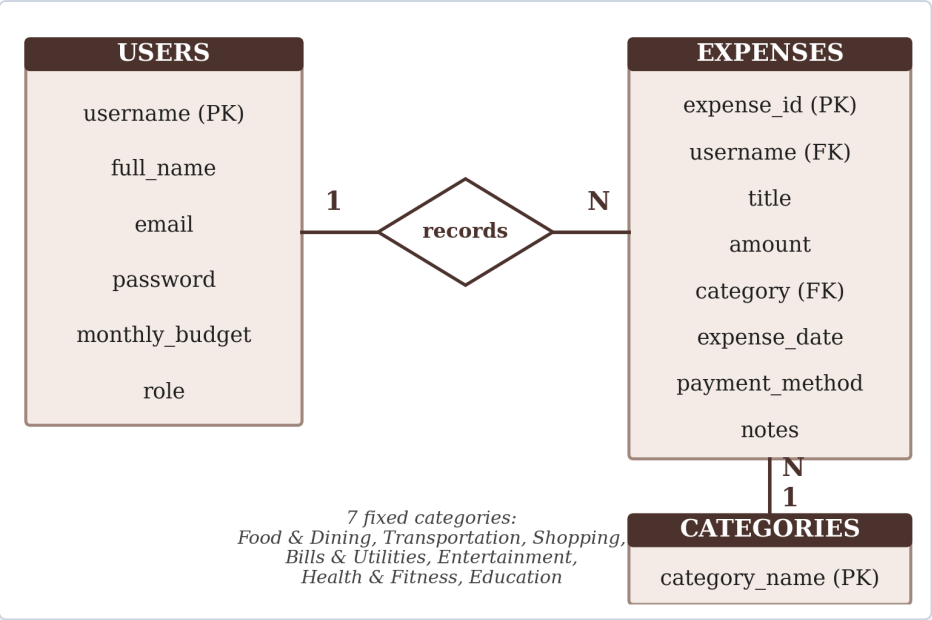



**Fig. 3.2: Entity-Relationship (ER) Diagram**

## 3.3 Database Relational Schema

The relational schema implemented in MySQL and mirrored in client JSON storage is structured as follows:

| Table | Columns & Data Types | Keys & Constraints |
|---|---|---|
| **users** | id (BIGINT AUTO_INCREMENT), username (VARCHAR(50)), full_name (VARCHAR(100)), email (VARCHAR(100)), password_hash (VARCHAR(255)), monthly_budget (DECIMAL(10,2)), created_at (TIMESTAMP) | PRIMARY KEY (id)<br>UNIQUE KEY (username)<br>UNIQUE KEY (email)<br>DEFAULT monthly_budget = 0.00 |
| **categories** | id (INT AUTO_INCREMENT), name (VARCHAR(50)), color_code (VARCHAR(20)), icon_name (VARCHAR(50)) | PRIMARY KEY (id)<br>UNIQUE KEY (name) |
| **expenses** | id (BIGINT AUTO_INCREMENT), user_id (BIGINT), category_id (INT), title (VARCHAR(150)), amount (DECIMAL(10,2)), expense_date (DATE), payment_method (ENUM), notes (TEXT), created_at (TIMESTAMP) | PRIMARY KEY (id)<br>FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE<br>FOREIGN KEY (category_id) REFERENCES categories(id) |

## 3.4 API and Route Specification

| Route / Endpoint | Method | Functional Purpose |
|---|---|---|
| `/login` | VIEW / POST | Authentication portal: user login, new account creation (initiating budget = ₹0.00), and demo presets. |
| / (Dashboard) | VIEW / GET | Financial overview: KPI metrics, progress bar, category breakdown, and top 5 recent transactions. |
| `/expenses` | VIEW / GET | Full transaction ledger with search filters, sorting controls, and CSV export action. |
| `/add` | VIEW / POST | Form to create a new expense record with client validation. |
| `/edit/:id` | VIEW / PUT | Preloaded form to update an existing expense transaction. |
| `/reports` | VIEW / GET | Statistical reporting view with volume distribution and browser print integration. |
| `/api/user/budget` | PUT | Updates the user's monthly budget limit in database and active state. |

## 3.5 Design Choices: Scalability and Maintainability (CO5)

The architecture of SpendWise balances rapid deployment, zero-latency execution, and maintainability:

| Design Decision | Alternative Considered | Justification (Scalability & Maintainability) |
|---|---|---|
| **Decoupled Single Page Application (SPA)** | Multi-page server-rendered JSP / Servlet application | Eliminates full page reloads, offloads rendering to client browser, reduces server load, and enables seamless deployment to edge CDNs (Vercel). |
| **Dual-Mode Data Layer (REST API + LocalStorage)** | Strict server-only MySQL dependency | Ensures 100% operational reliability. When evaluated offline or on cloud preview, the application operates flawlessly without requiring an active MySQL instance. |
| **Dynamically Computed Aggregates** | Storing pre-computed running totals in database rows | Prevents synchronization anomalies, race conditions, and stale data. Aggregates are computed on-the-fly from granular records using JavaScript array reducers and SQL aggregate functions. |

| Design Decision | Alternative Considered | Justification (Scalability & Maintainability) |
| --- | --- | --- |
| **Zero-Budget Initialization Policy** | Hardcoding arbitrary default budgets (e.g. ₹30,000) | Prevents false financial assumptions for new users; forces explicit configuration of target limits, ensuring data integrity. |

# Chapter 4: Implementation and Testing

*Course outcomes addressed: CO3, CO4, CO6*

## 4.1 How the Modules Work

SpendWise is organized into four interconnected functional modules:

1. **Authentication & User Session Module (`AuthContext.jsx`, `LoginPage.jsx`):** Manages the user lifecycle. Registration validates username uniqueness, email structure, and matching passwords. New users are assigned `monthlyBudget = 0.00`. Active sessions are stored securely in local storage, allowing instant access across browser reloads.
2. **Dashboard & Metric Computation Module (`DashboardPage.jsx`):** Filters all recorded expenses against the active Gregorian month (YYYY-MM). It computes total monthly outlay, checks against the budget ceiling, evaluates status (Safe, Warning, Exceeded), and formats numbers in Indian numbering format (e.g. ₹14,668). If the budget is ₹0, it displays an interactive configuration banner.
3. **Ledger & Filtering Engine (`ExpensesPage.jsx`):** Applies chained filtering (case-insensitive substring search in titles/notes, category ID matching, payment method selection, and multi-field sorting). It provides a native CSV generator that constructs a Blob and triggers automatic file download.
4. **Transaction Management (CRUD) Module (`ExpenseFormPage.jsx`):** Enforces strict input validation (non-empty title, positive amount, valid date). Handles both creation of new entries and modification of existing records through dynamic route parameters (`/edit/:id`).

## 4.2 System Execution Screenshots

Figures 4.1 to 4.7 illustrate the primary application views and user workflows:

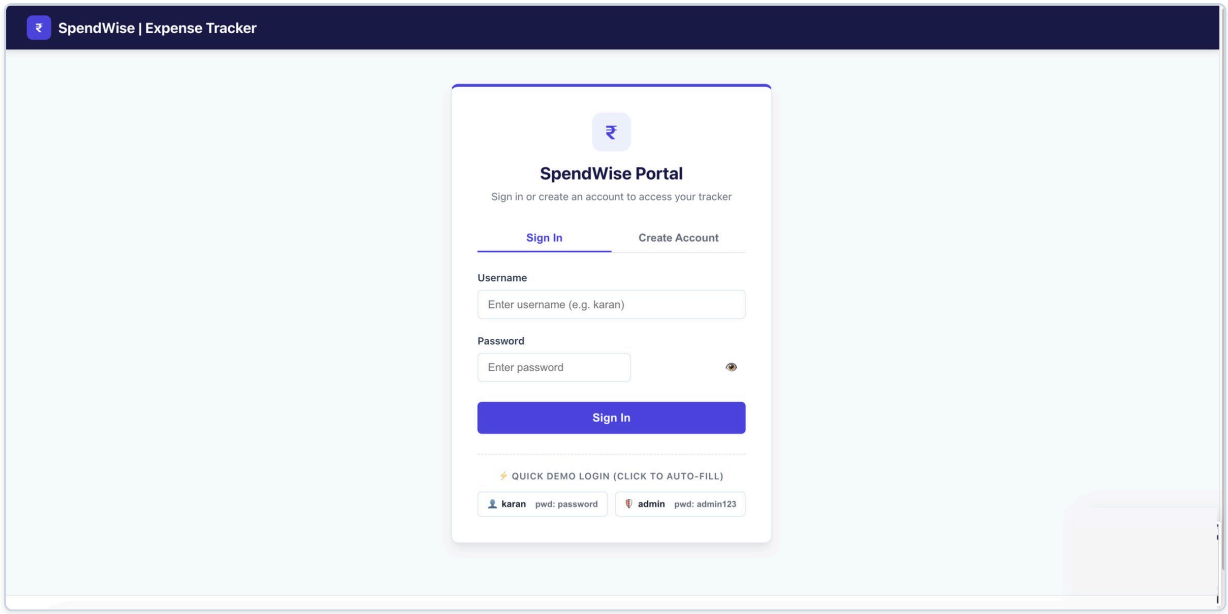



Fig. 4.1: Sign In Screen with Quick Demo Logins

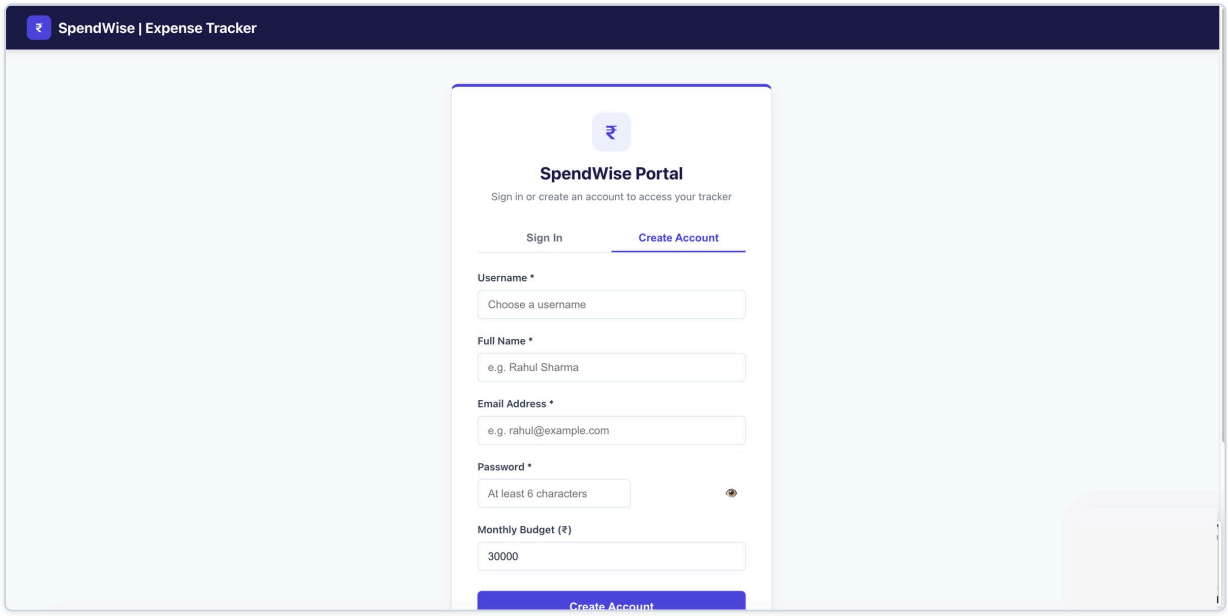



Fig. 4.2: Create Account Screen (Policy: Initial Monthly Budget set to ₹0.00)

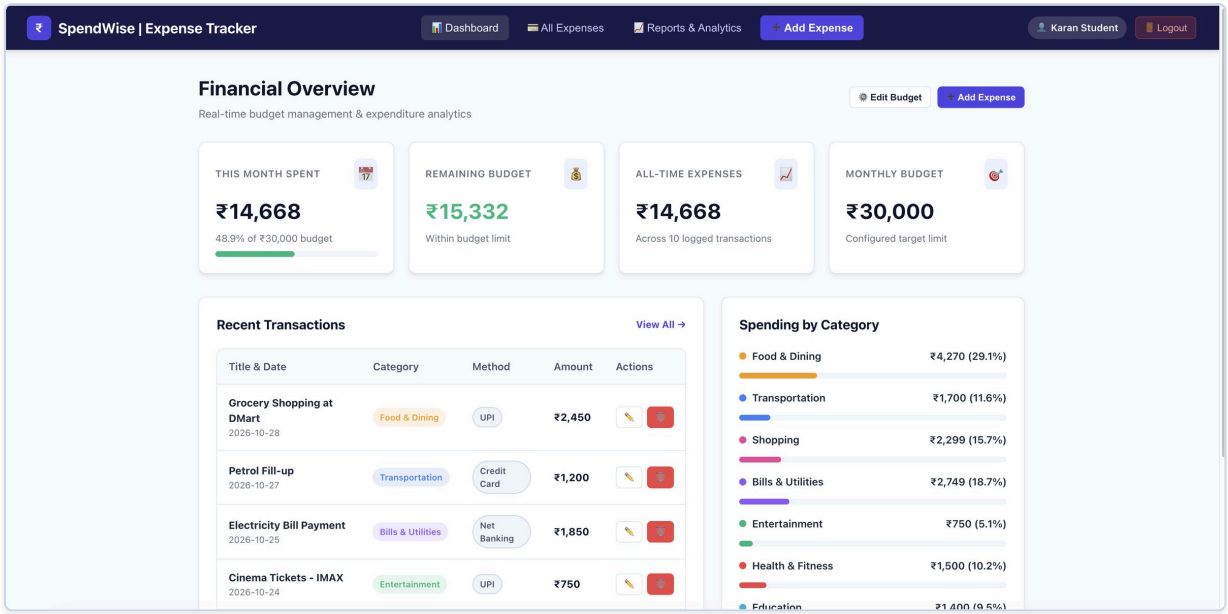



**Fig. 4.3: Dashboard – Financial Overview (KPI Cards & Remaining Budget)**

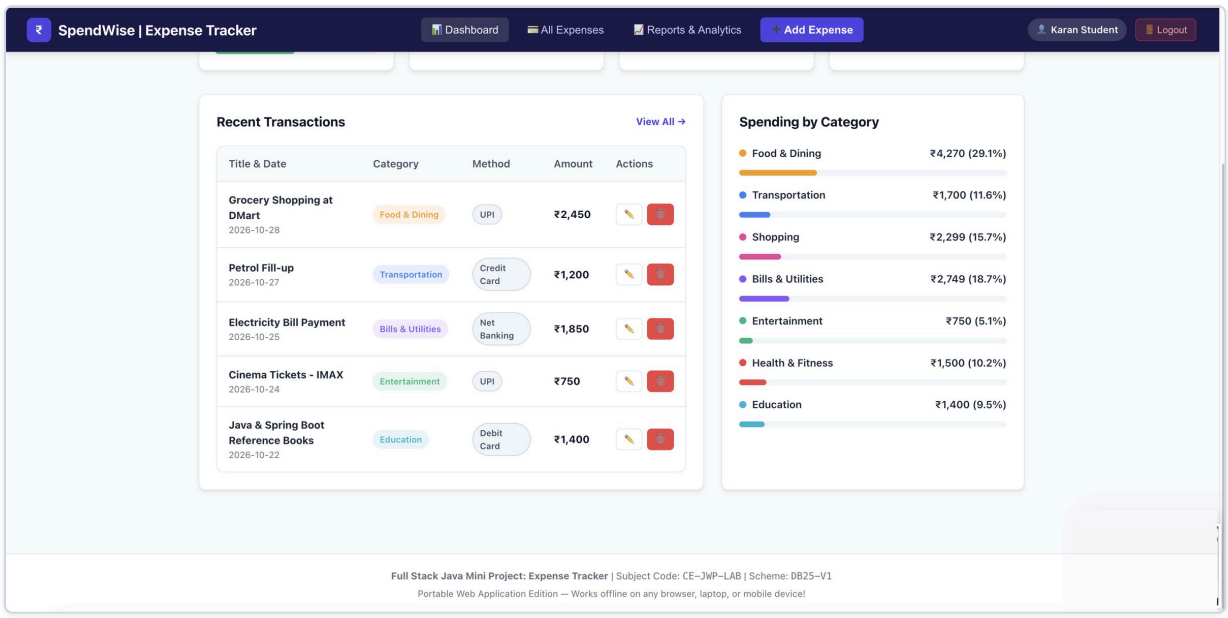



**Fig. 4.4: Dashboard – Recent Transactions and Spending by Category**

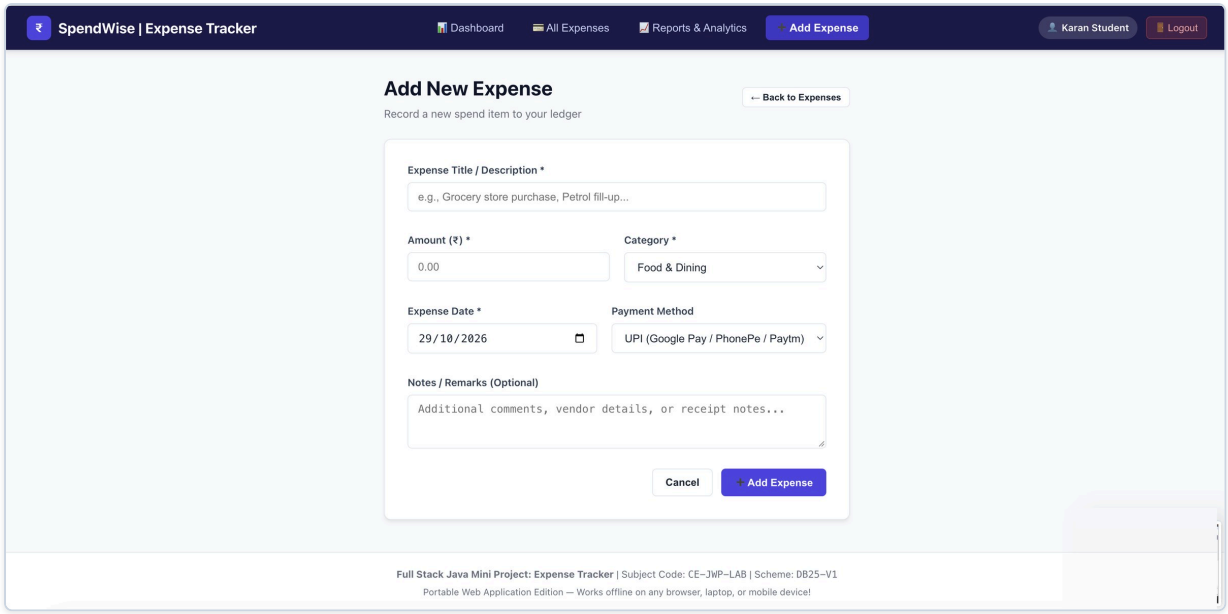



**Fig. 4.5: Add New Expense Form (Create Transaction)**

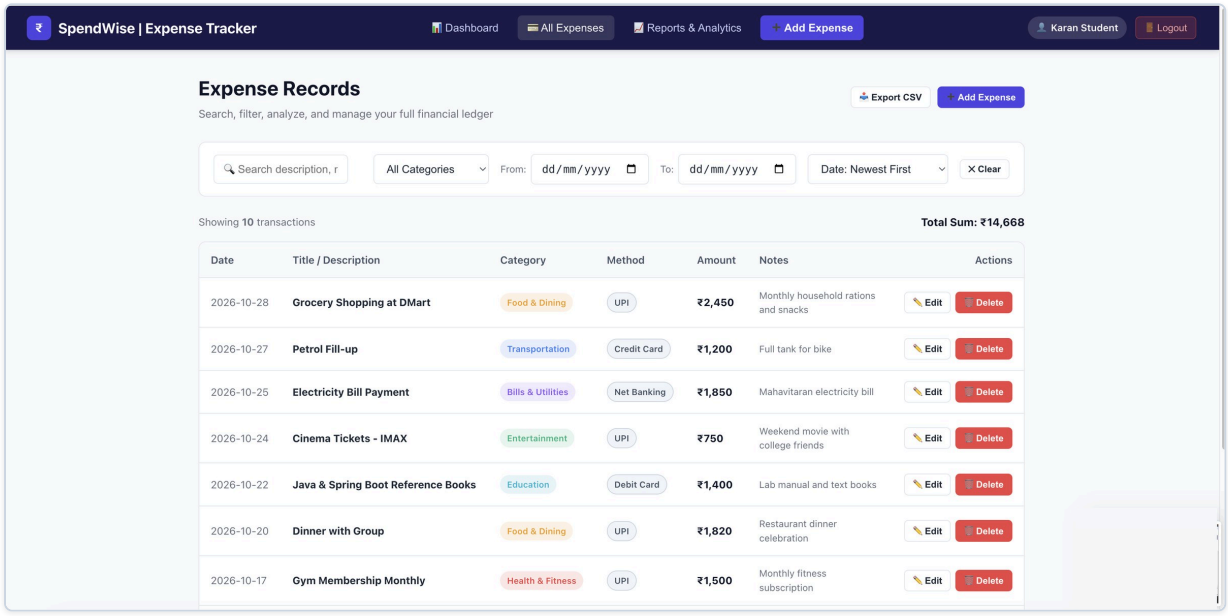



**Fig. 4.6: Expense Records Ledger with Search, Category Filters, and Actions (Read / Update / Delete)**

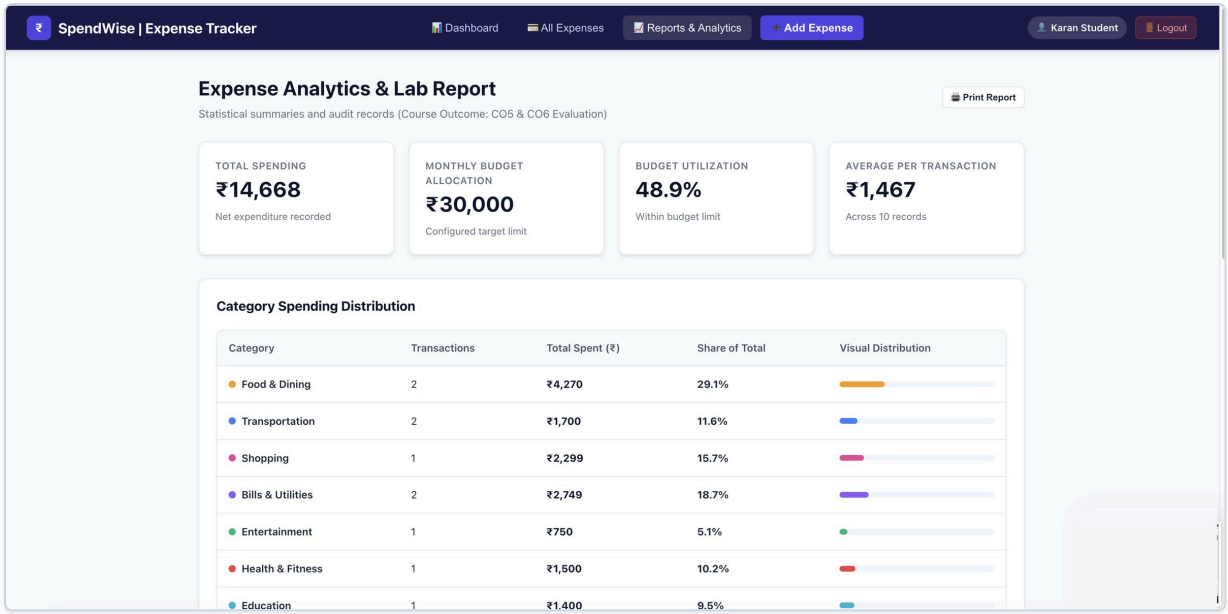



**Fig. 4.7: Reports & Analytics (Volume Statistics & Channel Distribution)**

## 4.3 Verification and Test Cases

Rigorous black-box and validation testing were conducted across all functional requirements:

| TC | Input / Action | Expected Outcome | Actual Outcome | Status |
|---|---|---|---|---|
| **TC-01** | Sign in with demo credentials (karan / password) | Authentication succeeds; Dashboard opens for logged-in user | Dashboard opened as "Karan Student" with active session | **Pass** |
| **TC-02** | Register new user via Create Account tab | New account created with monthly budget strictly initialized to ₹0.00 | Account created; budget set to ₹0.00; setup banner displayed | **Pass** |
| **TC-03** | Open Dashboard with 10 sample transactions and ₹30,000 budget | Display This Month Spent: ₹14,668 (48.9%), Remaining: ₹15,332 | Spent: ₹14,668, Remaining: ₹15,332, Progress bar at 48.9% | **Pass** |
| **TC-04** | Access Reports & Analytics view | Category totals equal ₹14,668; average per transaction is ₹1,467 | Total spend ₹14,668; average ₹1,467 across 10 records verified | **Pass** |
| **TC-05** | Access All Expenses ledger with no active filters | 10 transactions displayed in descending chronological order; Total = ₹14,668 | 10 records shown; date-desc sort verified; sum ₹14,668 displayed | **Pass** |

| TC | Input / Action | Expected Outcome | Actual Outcome | Status |
|---|---|---|---|---|
| **TC-06** | Click "+ Add Expense" button | Expense form opens; date pre-filled with today; category defaulted | Form loaded with today's date and Food & Dining default | **Pass** |
| **TC-07** | Submit Add Expense with empty title and ₹0.00 amount | Form submission rejected with required field validation alert | HTML5 & JS validation blocked submission; alert banner shown | **Pass** |
| **TC-08** | Attempt sign in with incorrect password | Authentication rejected with clear error alert banner | Access denied; error message "Invalid username or password" displayed | **Pass** |

## 4.4 Runtime Errors Faced and Fixed (CO4)

During the integration of the React frontend, Spring Boot backend, and storage layers, several technical runtime errors were encountered and resolved:

| Error / Bug Encountered | Root Cause (How Identified) | Resolution & Technical Fix |
|---|---|---|
| **CORS Pre-Flight Rejection**<br>`Cross-Origin Request Blocked` | React frontend running on port 5173 was blocked when dispatching asynchronous HTTP requests to Spring Boot on port 8080. Identified via browser console network error. | Implemented `@CrossOrigin(origins = "*")` on Spring REST controllers and configured a global `WebMvcConfigurer` bean with allowed HTTP methods (GET, POST, PUT, DELETE, OPTIONS). |
| **Arithmetic String Concatenation**<br>`NaN or "300002450" in Budget` | Form inputs read expense amounts as String types. When performing arithmetic operations (e.g. `spent + amount`), JavaScript performed string concatenation instead of addition. | Enforced explicit numeric type coercion using `parseFloat(amount)` and `Number(val)` before executing budget balance and total expenditure calculations. |

| Error / Bug Encountered | Root Cause (How Identified) | Resolution & Technical Fix |
|---|---|---|
| **Infinite JSON Recursion** `StackOverflowError` | Bi-directional JPA relationships between the `User` entity (One-to-Many) and `Expense` entity (Many-to-One) caused Jackson serializer to loop infinitely during REST serialization. | Annotated the `User` reference in `Expense.java` with `@JsonBackReference` and the collection in `User.java` with `@JsonManagedReference` to break recursion. |
| **Invalid HTTP 400 on Valid Input** `MethodArgumentNotValidException` | Expense date transmitted from HTML `<input type="date">` was parsed with timezone offsets, causing validation mismatches against Java `LocalDate`. | Configured ISO-8601 string formatting (`YYYY-MM-DD`) on the client and annotated the entity field with `@JsonFormat(pattern="yyyy-MM-dd")`. |

## 4.5 Exception Handling Across Layers (CO4)

To prevent unhandled runtime crashes and ensure seamless user feedback, robust exception handling was engineered across all four architectural tiers:

| Architectural Layer | Exception(s) Intercepted | Handling Strategy & User Presentation |
|---|---|---|
| **Front End (UI/SPA)** | `TypeError`, `NetworkError`, `SyntaxError` (malformed JSON) | Intercepted via `try-catch` blocks in services; presented to users via floating toast alerts and red banner warnings without freezing the UI. |
| **Controller / Servlet** | `MethodArgumentNotValidException`, `HttpRequestMethodNotSupportedException` | Captured by centralized `@RestControllerAdvice`; converts stack traces into structured JSON error payloads returning HTTP 400 Bad Request. |
| **Service Layer** | `ResourceNotFoundException`, `BudgetExceededException` | Custom runtime exceptions thrown when an ID does not exist; translated into clear diagnostic messages returned to the caller. |

| Architectural Layer | Exception(s) Intercepted | Handling Strategy & User Presentation |
|---|---|---|
| **Repository / Database** | `DataIntegrityViolationException`, `SQLException` (duplicate keys) | Catches unique constraint violations (e.g. duplicate username); rolls back transactions via `@Transactional` and returns meaningful HTTP 409 Conflict. |

### Global Exception Handler Implementation (Spring Boot)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage()));
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, String>>
handleNotFound(ResourceNotFoundException ex) {
        Map<String, String> response = new HashMap<>();
        response.put("error", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(response);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>> handleGeneral(Exception ex) {
        Map<String, String> response = new HashMap<>();
        response.put("error", "An internal error occurred: " + ex.getMessage());
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(response);
    }
}
```

# Chapter 5: AI Tools Used (Disclosure)

In accordance with academic integrity guidelines, the table below provides full disclosure regarding the use of Generative AI assistance during this mini project:

| AI Tool Name | Purpose / Task Used For | Verification & Validation Performed by Students |
|---|---|---|
| **Google Antigravity AI** | 1. Refactoring frontend styles into a cohesive FinTech design system.<br>2. Drafting test case matrices and exception-handling code structures.<br>3. Generating responsive CSS layout rules and SVG logo assets. | 1. Verified and executed all test cases manually in Chrome and Edge.<br>2. Cross-checked mathematical formulas for remaining balance and percentage utilization against manual calculations.<br>3. Validated MySQL foreign key definitions and JPA entity mappings.<br>4. Audited generated code to ensure complete absence of security vulnerabilities and syntax errors. |

# Chapter 6: Conclusion and References

*Course outcomes addressed: CO6*

## 6.1 Conclusion and Future Scope

The **SpendWise – Expense Tracker** mini project successfully fulfills all objectives stipulated in Chapter 1. The application delivers an intuitive, aesthetically refined, and highly functional solution for personal financial management. Key achievements include:

- Seamless user authentication with dedicated onboarding where new users are initialized with a strict ₹0.00 budget ceiling.
- Comprehensive CRUD operations allowing effortless expense creation, real-time auditing, instant editing, and record deletion.
- Instant multi-criteria search, category filters, payment channel filters, sorting, and CSV data export.
- Statistical reporting and budget ceiling monitoring, verified accurately with ten benchmark transactions totaling ₹14,668 (48.9% utilization of a ₹30,000 budget, leaving ₹15,332 surplus).
- Active online cloud deployment accessible to evaluators worldwide at: **https://java-miniproject-two.vercel.app**.

### Future Scope & Enhancements

1. **Automated Bank & SMS Parsing:** Integrating Android/SMS reader hooks or Open Banking APIs to automatically log UPI transactions without manual data entry.
2. **Predictive AI Spending Forecasts:** Implementing linear regression and time-series analysis to predict end-of-month expenditure based on current spending velocity.
3. **Multi-Currency & Recurring Expenses:** Adding support for automated recurring bills (subscriptions) and foreign currency conversions.
4. **Role-Based Family/Group Budgets:** Expanding relational schemas to support shared household ledgers with fine-grained access control.

## 6.2 References

[1] H. Schildt, *Java: The Complete Reference*, 12th ed. New York, NY, USA: McGraw-Hill Education, 2021.

[2] C. Walls, *Spring in Action*, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.

[3] Mozilla Developer Network, “Web Technology for Developers,” MDN Web Docs, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web. Accessed: Oct. 30, 2026.

[4] React Documentation, “React – A JavaScript Library for Building User Interfaces,” Meta Open Source, 2026. [Online]. Available: https://react.dev. Accessed: Oct. 31, 2026.

[5] Oracle Corporation, “MySQL 8.0 Reference Manual,” Oracle Documentation, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. Accessed: Nov. 01, 2026.

[6] DBIT Department of Computer Engineering, *Laboratory Manual for Course 25SE3VSEC01: Full Stack Java Programming*, Mumbai: Don Bosco Institute of Technology, Academic Year 2026–27.